

Python and Data Engineering Interview Questions & Answers

1. What is Serialization in Python?

Serialization is the process of converting Python objects into a format that can be stored on disk or transmitted over a network[1]. It transforms an in-memory object structure into a byte stream, making it portable and persistent.

Key characteristics:

- Converts complex data structures (objects, dictionaries, lists) into a standardized format
- Enables data persistence and inter-process communication
- Commonly used with pickle, JSON, and other serialization libraries
- Reverse process is called deserialization (converting bytes back to objects)

Example use cases:

- Saving machine learning model parameters
 - Caching computation results
 - Sending objects over APIs
 - Storing configuration data
-

2. What is the Use of a Pickle File?

Pickle is Python's built-in serialization module that converts Python objects into byte streams[1]. The .pkl or .pickle extension denotes these serialized files.

Primary uses:

- **Model persistence:** Save trained ML models (scikit-learn, TensorFlow models)
- **Data caching:** Store intermediate computation results to avoid reprocessing
- **Object preservation:** Save complex Python objects with their state intact
- **Data interchange:** Share Python objects between different scripts or processes

Advantages:

- Native Python support (part of standard library)
- Preserves Python object types and structures
- Efficient binary format

Disadvantages:

- Security risk: pickle can execute arbitrary code during deserialization
- Not human-readable
- Python version compatibility issues
- Not suitable for cross-language communication

Best practice: Never unpickle data from untrusted sources.

```
import pickle
```

Saving objects to pickle file

```
data = {"model": "trained_model", "accuracy": 0.95}
with open("data.pkl", "wb") as f:
    pickle.dump(data, f)
```

Loading from pickle file

```
with open("data.pkl", "rb") as f:
    loaded_data = pickle.load(f)
```

3. How Will You Save Data from Python into JSON and CSV Formats?

JSON Format

JSON (JavaScript Object Notation) is a human-readable, text-based format ideal for structured data and APIs[1].

```
import json
```

Dictionary to JSON file

```
data = {
    "name": "Anil",
    "role": "Data Engineer",
    "skills": ["Python", "Spark", "SQL"],
    "experience": 15
}
```

Write to JSON

```
with open("data.json", "w") as f:
    json.dump(data, f, indent=4)
```

Read from JSON

```
with open("data.json", "r") as f:
    loaded_data = json.load(f)
```

JSON string conversion

```
json_string = json.dumps(data, indent=2)
```

Advantages: Human-readable, lightweight, supports nested structures, universal language support

CSV Format

CSV (Comma-Separated Values) is tabular data format commonly used for spreadsheets and data analysis[1].

```
import csv
import pandas as pd
```

Method 1: Using csv module

```
data = [
    ["Name", "Role", "Experience"],
    ["Anil", "Data Engineer", "15"],
    ["Priya", "Backend Engineer", "12"]
]

with open("data.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerows(data)
```

Read CSV

```
with open("data.csv", "r") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

Method 2: Using pandas (recommended for data analysis)

```
df = pd.DataFrame(data[1:], columns=data[0])
df.to_csv("data.csv", index=False)
```

Read CSV with pandas

```
df = pd.read_csv("data.csv")
```

Advantages: Wide compatibility with spreadsheet applications, simple format, efficient for tabular data

Key difference: JSON preserves data types and nested structures, while CSV is flat and all values are strings.

4. Remove Duplicates and Count Elements from a List Without Using Inbuilt Methods

Problem: Use only primitive operations, no set(), len(), or count() methods.

```
def remove_duplicates_and_count(lst):  
    """
```

Remove duplicates and count elements without inbuilt methods.

Time Complexity: $O(n^2)$

Space Complexity: $O(n)$

```
    """
```

```
    unique_list = []
```

```
    counts = []
```

```
        # Remove duplicates -  $O(n^2)$  approach
```

```
        i = 0
```

```
        while i < len(lst):
```

```
            found = False
```

```
            j = 0
```

```
            while j < len(unique_list):
```

```
                if lst[i] == unique_list[j]:
```

```
                    found = True
```

```
                    break
```

```
                j = j + 1
```

```
            if not found:
```

```
                unique_list.append(lst[i])
```

```
            i = i + 1
```

```
        # Count elements -  $O(n^2)$  approach
```

```
        i = 0
```

```
        while i < len(unique_list):
```

```
            count = 0
```

```
            j = 0
```

```
            while j < len(lst):
```

```
                if unique_list[i] == lst[j]:
```

```
                    count = count + 1
```

```
                j = j + 1
```

```
counts.append(count)
i = i + 1

return unique_list, counts
```

Usage

```
numbers = [1, 2, 2, 3, 3, 3, 4, 4, 4, 4]
unique, counts = remove_duplicates_and_count(numbers)
print(f"Unique: {unique}")
print(f"Counts: {counts}")
```

Output: Unique: [1, 2, 3, 4], Counts: [1, 2, 3, 4]

Optimized approach using dictionary simulation:

```
def remove_duplicates_optimized(lst):
    """
    Using hash map simulation - O(n) time complexity
    """
    count_map = [] # List of [value, count] pairs
```

```
    for element in lst:
        found = False
        for i in range(len(count_map)):
            if count_map[i][0] == element:
                count_map[i][1] = count_map[i][1] + 1
                found = True
                break

        if not found:
            count_map.append([element, 1])

    unique_list = [item[0] for item in count_map]
    counts = [item[1] for item in count_map]

    return unique_list, counts
```

5. How Do You Connect to an SQL DB Using Python? What Library Do You Use?

Primary Libraries

SQLAlchemy: Most popular ORM (Object-Relational Mapping) framework[1]
`from sqlalchemy import create_engine, text`

Connection string:
**dialect+driver://user:password@host:port/
database**

```
engine = create_engine('postgresql://user:password@localhost:5432/mydb')
```

Execute query

```
with engine.connect() as connection:  
    result = connection.execute(text("SELECT * FROM users"))  
    for row in result:  
        print(row)
```

psycopg2: PostgreSQL specific adapter
`import psycopg2`

```
connection = psycopg2.connect(  
    host="localhost",  
    database="mydb",  
    user="postgres",  
    password="password",  
    port=5432  
)  
  
cursor = connection.cursor()  
cursor.execute("SELECT * FROM users")  
rows = cursor.fetchall()  
  
cursor.close()  
connection.close()
```

MySQL connector:
`import mysql.connector`

```
connection = mysql.connector.connect(  
    host="localhost",  
    user="root",  
    password="password",  
    database="mydb"  
)
```

```
cursor = connection.cursor()
cursor.execute("SELECT * FROM users")
```

pandas (for data analysis):

```
import pandas as pd
from sqlalchemy import create_engine
```

```
engine = create_engine('postgresql://user:password@localhost:5432/mydb')
df = pd.read_sql("SELECT * FROM users", engine)
```

Connection Best Practices

- Use connection pooling for production environments
- Parameterize queries to prevent SQL injection
- Handle connection timeouts and errors gracefully
- Close connections explicitly or use context managers

6. Why Can't You Use SQL DB for Semantic Search?

Traditional SQL databases are fundamentally designed for **exact keyword matching** and structured data queries, not semantic similarity[1].

Limitations:

1. No semantic understanding: SQL uses lexical matching (exact word/substring match). It doesn't understand meaning or similarity between concepts. Query "car" won't match "automobile."

2. Fixed schema structure: SQL requires predefined column types. Semantic information requires high-dimensional vector embeddings which SQL cannot efficiently store or query.

3. Inefficient with high-dimensional data: SQL uses B-tree indexes optimized for scalar values, not vectors. Querying millions of dimensions creates cartesian products, causing exponential computational complexity.

4. No approximate nearest neighbor (ANN) search: Semantic search requires finding the K most similar items to a query. SQL is designed for exact matching, not similarity ranking.

5. Performance degradation: SQL distance calculations (Euclidean, cosine) across millions of vectors would require scanning entire tables, making real-time semantic search impractical.

Example problem:

-- Inefficient and won't find semantic matches

```
SELECT * FROM documents WHERE content LIKE '%renewable%'
```

-- Won't find: "clean energy", "solar power", "sustainable power"

Solution: Vector databases (Pinecone, Weaviate, Qdrant, Milvus) are purpose-built for semantic search using ANN algorithms (HNSW, IVF, LSH) that efficiently find similar vectors in milliseconds.

7. What is Preprocessing of Data?

Data preprocessing is the process of cleaning and transforming raw data into a format suitable for machine learning models and analysis[1].

Key preprocessing steps:

Step	Description	Example
Data Cleaning	Handle missing values, outliers, duplicates	Remove null values, flag anomalies
Data Transformation	Convert data to appropriate format/scale	Normalize to 0-1, standardize to z-score
Feature Engineering	Create new features from existing data	Combine date columns into age feature
Data Encoding	Convert categorical to numerical	One-hot encoding, label encoding
Data Validation	Check data quality and consistency	Verify data types, check constraints

Common preprocessing techniques:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler
```

Load data

```
df = pd.read_csv('data.csv')
```

Handle missing values

```
df.fillna(df.mean(), inplace=True) # Mean imputation
df.dropna(inplace=True) # Remove rows with nulls
```

Remove duplicates

```
df.drop_duplicates(inplace=True)
```


Outlier detection

```
Q1 = df['column'].quantile(0.25)
Q3 = df['column'].quantile(0.75)
IQR = Q3 - Q1
df = df[(df['column'] >= Q1 - 1.5*IQR) & (df['column'] <= Q3 + 1.5*IQR)]
```

Normalization (scaling to 0-1)

```
scaler = MinMaxScaler()
df['scaled_column'] = scaler.fit_transform(df[['column']])
```

Standardization (z-score normalization)

```
scaler = StandardScaler()
df['standardized_column'] = scaler.fit_transform(df[['column']])
```

Categorical encoding

```
df['category_encoded'] = pd.factorize(df['category'])[0] # Label encoding
df = pd.get_dummies(df, columns=['category']) # One-hot encoding
```

Why it matters:

- Poor quality data leads to poor model performance
- Algorithms assume data within certain ranges or distributions
- Missing values can bias results
- Standardized data improves convergence in neural networks

8. Why Use Regex Instead of NLP?

Regex (Regular Expressions) and NLP (Natural Language Processing) serve different purposes and have complementary strengths[1].

When to use Regex:

Use Case	Advantage
Pattern matching	Fast, deterministic, exact patterns (emails, phone numbers, URLs)
Structured text extraction	Simple rules, no ML training needed
Text preprocessing	Remove special characters, whitespace normalization
High precision	Exact matches with zero false positives
Performance critical	Millisecond latency requirements
Small datasets	No need for large training corpora

Example Regex use cases:

```
import re
```

Email validation

```
email_pattern = r'^[1]+@[a-zA-Z0-9-]+\.[a-zA-Z]{2,}$'
if re.match(email_pattern, "anil@example.com"):
    print("Valid email")
```

Phone number extraction

```
phones = re.findall(r'\d{3}-\d{3}-\d{4}', text)
```

URL extraction

```
urls = re.findall(r'https?:\/\/[^\s]+', text)
```

Remove HTML tags

```
clean_text = re.sub(r'<[^>]+>', '', html_content)
```

When to use NLP:

Use Case	Advantage
Semantic understanding	Understands meaning beyond keywords
Ambiguity resolution	Handles context and multiple interpretations
Sentiment analysis	Identifies emotional tone accurately
Named entity recognition	Extracts people, organizations, locations
Complex patterns	Learns from data rather than hardcoded rules
Language variations	Handles synonyms, slang, abbreviations

```
from nltk.sentiment import SentimentIntensityAnalyzer
from spacy import load
```

Sentiment analysis

```
analyzer = SentimentIntensityAnalyzer()
score = analyzer.polarity_scores("This product is amazing!")
```

Named entity recognition

```
nlp = load('en_core_web_sm')
doc = nlp("Anil works at JP Morgan Chase")
for ent in doc.ents:
    print(f"{ent.text}: {ent.label_}")
```

Decision framework:

- **Structured, deterministic patterns** → Regex (faster, simpler)
- **Semantic, contextual, ambiguous text** → NLP (more powerful)
- **Best practice:** Combine both (regex for preprocessing, NLP for analysis)

9. What is Top-P?

Top-P (nucleus sampling) is a text generation strategy that controls randomness and diversity in language model outputs[1].

Definition:

Top-P selects the smallest set of tokens whose cumulative probability reaches P (typically 0.9), then randomly samples from this filtered set. It adaptively adjusts the number of candidates based on probability distribution.

How it works:

Original tokens: [A:0.5, B:0.3, C:0.15, D:0.04, E:0.01]

With Top-P=0.9:

- A (0.5): cumulative = 0.5
- B (0.3): cumulative = 0.8
- C (0.15): cumulative = 0.95 ← Exceeds 0.9, so exclude C, D, E
Sample from: {A, B} only

Comparison with other sampling methods:

Method	Description	Use Case
Top-K	Select K most probable tokens	Limits diversity
Top-P	Dynamic selection based on cumulative probability	Better quality, natural diversity
Temperature	Controls distribution "sharpness"	Complementary to Top-P
Greedy	Always pick highest probability token	Deterministic, repetitive

Parameter effects:

- **Top-P = 0.9:** Balance between quality and diversity (typical)
- **Top-P = 0.5:** More conservative, focused outputs
- **Top-P = 1.0:** No filtering, allows all tokens
- **Top-P = 0.1:** Highly restrictive, repetitive

Python example (with OpenAI):

```
import openai

response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[{"role": "user", "content": "Write a story"}],
    top_p=0.9, # Nucleus sampling
    temperature=0.7, # Adjust randomness
    max_tokens=200
)
```

Use cases:

- Creative writing (higher Top-P for diversity)
 - Chatbots (moderate Top-P for natural responses)
 - Code generation (lower Top-P for correctness)
 - Summarization (low Top-P for consistency)
-

10. What is an Embedding? How Many Dimensions Does an OpenAI Embedding Have?

What is an Embedding?

An embedding is a numerical representation of text (or any data) as a vector in continuous high-dimensional space[1]. It captures semantic meaning: similar texts have similar embeddings.

Key characteristics:

- Converts discrete tokens/words into dense real-valued vectors
- Captures semantic and syntactic relationships
- Enables mathematical operations (similarity, clustering, classification)
- Foundation of modern NLP and semantic search

Visualization example:

Text: "Machine Learning"

Embedding: [-0.234, 0.891, -0.456, 0.123, ..., 0.678]

(1,536 or 3,072 dimensions)

Similar texts:

"Deep Learning" → Similar embedding (high cosine similarity)

"Artificial Intelligence" → Very similar embedding

"Cat" → Very different embedding (low cosine similarity)

OpenAI Embedding Dimensions

text-embedding-3-small: [1]

- Default dimensions: **1,536** (same as Ada v2)
- Configurable: Can reduce to **512** without performance loss
- Use case: Latency-sensitive applications, cost optimization

text-embedding-3-large: [1]

- Default dimensions: **3,072**
- Configurable: Can reduce to 256 without impacting accuracy
- Remarkable: 256-dimensional version can outperform Ada v2's 1536 dimensions (6x reduction)
- Use case: High accuracy requirements, production-grade semantic search

Comparison:

Model	Default Dimensions	Min Dimensions	Use Case
text-embedding-3-small	1,536	512	Latency, cost
text-embedding-3-large	3,072	256	Accuracy, RAG

Python usage:

```
import openai
```

Default dimensions

```
response = openai.Embedding.create(  
    model="text-embedding-3-large",  
    input="Machine Learning"  
)  
embedding = response['data'][0]['embedding']  
print(f"Dimensions: {len(embedding)}") # 3,072
```

Reduced dimensions

```
response = openai.Embedding.create(  
    model="text-embedding-3-large",  
    input="Machine Learning",  
    dimensions=256 # Reduce to 256 for efficiency  
)  
embedding = response['data'][0]['embedding']  
print(f"Dimensions: {len(embedding)}") # 256
```

11. Difference Between text-embedding-3-small vs text-embedding-3-large

Detailed Comparison

Aspect	text-embedding-3-small	text-embedding-3-large
Default Dimensions	1,536	3,072
Min Dimensions	512	256
Latency	Faster (lower dimensions)	Slower (higher dimensions)
Storage	Lower (~4.9 KB per vector)	Higher (~9.8 KB per vector)
Accuracy	Good for most tasks	Excellent for complex queries
Cost	Lower per 1M tokens	Higher per 1M tokens
Recommended Use	Real-time systems, ChatOps	Production RAG, semantic search
Performance Trade-off	Optimized for speed	Optimized for accuracy

When to Use Each

text-embedding-3-small is ideal for:

- Real-time applications with latency constraints
- Cost-sensitive projects
- Quick prototyping and MVPs
- High-volume embedding requests (chatbots, logs)
- Applications where 512-dim is sufficient accuracy

Fast, cost-effective embedding

```
embedding = openai.Embedding.create(
model="text-embedding-3-small",
input="Query text",
dimensions=512 # Further optimize
)
```

text-embedding-3-large is ideal for:

- Production-grade RAG systems[1]
- Complex semantic queries requiring high accuracy

- Document retrieval where relevance matters
- Enterprise applications
- Where performance matters more than cost

High-accuracy embedding

```
embedding = openai.Embedding.create(
model="text-embedding-3-large",
input="Complex query about renewable energy policies",
dimensions=256 # Sufficient with 256-dim encoding
)
```

Key Insight

The breakthrough in text-embedding-3-large is **dimension reduction capability**. A 256-dimensional embedding from text-embedding-3-large can outperform 1536-dimensional Ada v2 embeddings, offering 6x storage savings with better accuracy[1].

12. Why Do You Need a Vector DB?

Traditional databases cannot efficiently handle semantic search on embeddings. Vector databases are purpose-built for this use case[1].

Limitations of SQL Databases

Problem 1: Inefficient similarity search

- SQL uses B-tree indexes for scalar values, not vectors
- Finding K most similar vectors requires scanning entire table
- Distance calculations create exponential computational complexity
- Infeasible for millions+ vectors

Problem 2: Wrong data structure

- SQL optimized for exact matching, not similarity ranking
- No native support for high-dimensional vector operations
- CRUD operations on 3,072-dimensional vectors extremely slow

Problem 3: Scalability issues

- Real-time semantic search at scale requires specialized algorithms
- SQL can't use approximate nearest neighbor (ANN) algorithms
- Cannot maintain index consistency while serving queries

Vector DB Advantages

1. Optimized indexing algorithms:[1]

- HNSW (Hierarchical Navigable Small World)
- IVF (Inverted File)
- LSH (Locality Sensitive Hashing)
- Enable millisecond-scale similarity search on billions of vectors

2. Approximate Nearest Neighbor (ANN) search:[1]

- Trades perfect accuracy for ~99.9% accuracy with massive speed gains
- Search 1M vectors in ~10-50ms vs hours in SQL

3. Efficient similarity operations:[1]

- Native cosine similarity, Euclidean distance, dot product
- Batch operations optimized for vectors
- Metadata filtering alongside vector search

4. Scalability:

- Designed for billions of vectors
- Distributed architectures (sharding, replication)
- Real-time ingestion of new vectors

Vector DB Use Cases

Semantic Search → Find documents/images similar to query

Recommendation Systems → Find products/content similar to user preferences

Duplicate Detection → Identify near-duplicate items in dataset

Anomaly Detection → Find outliers in vector space

Image/Video Search → Find visually similar content

Generative AI/RAG → Retrieve relevant context for LLM

Question Answering → Find relevant documents for QA systems

Popular Vector Databases

Pinecone → Fully managed, serverless

Weaviate → Open-source, cloud option

Qdrant → High-performance, Russian open-source

Milvus → Scalable, Kubernetes-native

Chroma → Lightweight, embeddings-first

13. Why Are Chunking and Embeddings Required?

Chunking and embeddings are foundational techniques in Retrieval-Augmented Generation (RAG) systems[1][2].

The Problem They Solve

Without chunking/embeddings:

- Entire documents are too large to process
- LLMs have finite context windows (4K-128K tokens)
- Can't perform semantic search on raw documents
- Information retrieval is limited to keyword matching

Chunking: Breaking Documents into Semantic Units

What is chunking:[2]

Dividing large documents into manageable, semantically meaningful pieces.

Why chunking is necessary:

1. **Context window limits:** GPT-4 has 128K tokens; documents exceed this
2. **Semantic coherence:** Chunks should contain related information
3. **Embedding efficiency:** Smaller chunks create focused, searchable embeddings
4. **Retrieval accuracy:** Relevant chunk retrieval > full document retrieval

Chunking strategies:[2]

Simple fixed-size chunking

```
def fixed_chunking(text, chunk_size=500, overlap=50):
    chunks = []
    for i in range(0, len(text), chunk_size - overlap):
        chunks.append(text[i:i + chunk_size])
    return chunks
```

Semantic chunking (sentence-aware)

```
def semantic_chunking(text, threshold=0.5):
```

```
    """
```

```
    Split on sentence boundaries.
```

```
    Merge small segments that are similar.
```

```
    """
```

```
    sentences = text.split('. ')
    chunks = []
    current_chunk = sentences[0]
```

```
    for sentence in sentences[1:]:
        if len(current_chunk) < 500:
            current_chunk += '. ' + sentence
        else:
            chunks.append(current_chunk)
            current_chunk = sentence
```

```
    return chunks
```

Recursive chunking (hierarchical)

```
def recursive_chunking(text, separators=['\n\n', '\n', ' '],
chunk_size=500, overlap=50):
    """
```

```
    Split by paragraph, then sentence, then word.
    More intelligent than fixed-size.
    """
```

```
    good_splits = []
    separator = separators[-1]
```

```
    for _s in separators:
        if _s in text:
            separator = _s
            break

    if separator:
        splits = text.split(separator)
    else:
        splits = list(text)

    good_splits = [s for s in splits if len(s) > chunk_size]

    # Merge small chunks
    merge_good_splits = []
    for good_split in good_splits:
        if len(good_split) < chunk_size:
            if merge_good_splits:
                merge_good_splits[-1] += separator + good_split
            else:
                merge_good_splits.append(good_split)
        else:
            merge_good_splits.append(good_split)

    return merge_good_splits
```

Chunking best practices:[2]

- **Chunk size:** 200-1000 tokens (typical: 500)
- **Overlap:** 10-20% of chunk size for context preservation
- **Strategy:** Recursive/semantic preferred over fixed-size

- **Late chunking:** Advanced technique where embeddings created before splitting

Embeddings: Converting Text to Vectors

What embeddings do:

Transform text chunks into dense vectors capturing semantic meaning.

Why embeddings are necessary:

1. **Semantic search:** Find similar chunks without keyword matching
2. **Similarity ranking:** Rank chunks by relevance to query
3. **Vector DB compatibility:** Store in vector DB for fast retrieval
4. **LLM context:** Convert text to format for prompt augmentation

Embedding workflow in RAG:[1]

Raw Document

↓

Chunking (break into 500-token pieces with overlap)

↓

Embedding (convert each chunk to 1536/3072-dim vector)

↓

Vector DB (store vectors with metadata)

↓

User Query

↓

Query Embedding (embed query in same space)

↓

Similarity Search (find K nearest chunks)

↓

LLM Prompt (augment with retrieved chunks)

↓

Generated Response

Python example:

```
from openai import OpenAI
import json
```

```
client = OpenAI()
```

Step 1: Chunk document

```
document = "..." # Large document
chunks = semantic_chunking(document, chunk_size=500)
```

Step 2: Embed chunks

```
chunk_embeddings = []
for chunk in chunks:
    response = client.embeddings.create(
        model="text-embedding-3-small",
        input=chunk
```

```
)  
chunk_embeddings.append({  
    "text": chunk,  
    "embedding": response.data[0].embedding  
})
```

Step 3: Store in vector DB (Pinecone, Weaviate, etc.)

```
vector_db.upsert(chunk_embeddings)
```

Step 4: Query

```
query = "What are renewable energy policies?"  
query_embedding = client.embeddings.create(  
    model="text-embedding-3-small",  
    input=query  
)  
.data[0].embedding
```

Step 5: Find similar chunks (similarity search)

```
similar_chunks =  
vector_db.query(query_embedding,  
top_k=3)
```

Step 6: Augment LLM prompt

```
augmented_prompt = f"""  
Context:  
{similar_chunks}
```

```
Query: {query}  
"""
```

```
response = client.chat.completions.create(  
    model="gpt-4",  
    messages=[{"role": "user", "content": augmented_prompt}]  
)
```

Why both are essential:

- **Chunking alone:** Without embeddings, still relies on keyword search
 - **Embeddings alone:** Without chunking, context window limits and high dimensionality issues
 - **Together:** Perfect balance of semantic understanding and computational efficiency
-

14. Why Did You Use Similarity Search? Was It Always Accurate?

Similarity search enables semantic retrieval in RAG systems but has inherent accuracy limitations[1].

Why Similarity Search is Used

Advantages:[1]

1. **Semantic understanding:** Finds contextually relevant documents, not just keyword matches
2. **Fast retrieval:** Vector DB ANN algorithms enable sub-millisecond search on billions of vectors
3. **Handles synonyms:** "car" queries find "automobile" documents automatically
4. **Ranking capability:** Returns ranked results by relevance score
5. **Scalability:** Efficient even with massive document collections

Use case example:

Query: "renewable energy sources"

Keyword search finds:

- ✓ "renewable energy policies"
- ✓ "solar power generation"
- ✓ "wind energy systems"
- ✗ "sustainable power alternatives" (missed)
- ✗ "clean energy transition" (missed)

Similarity search finds:

- ✓ "renewable energy policies"
- ✓ "solar power generation"
- ✓ "wind energy systems"
- ✓ "sustainable power alternatives"
- ✓ "clean energy transition"
- ✓ Ranked by semantic relevance

Accuracy Limitations

1. Semantic drift in dense retrieval:[1]

Query embedding space may not align perfectly with document space.

Example:

Query: "How to treat a broken arm?"

Retrieved: "Arm wrestling techniques" (similar vector, wrong context)

2. Dense passage retrieval (DPR) has fundamental limits:

- Approximate nearest neighbor (ANN) algorithms sacrifice ~0.1-1% accuracy

- 99.9% accuracy acceptable for 1000+ results, critical for top-3
- High-dimensional vector spaces have counter-intuitive properties

3. Chunk boundary problems:

Important information split across chunk boundaries

Query: "renewable energy policy implementation"

Retrieved chunk: "...policy implications..." (partial match)

Relevant context: In next chunk (not retrieved)

4. Embedding model limitations:

All semantic information compressed into fixed dimensions

text-embedding-3-large: 3,072 dims for entire document

Nuanced meanings can get lost in dimensionality reduction

5. Out-of-distribution queries:

Query uses terminology different from training data

Query: "photovoltaic electricity generation"

Training data: "solar power"

Embeddings diverge significantly

Strategies to Improve Accuracy

1. Hybrid search (combining keyword + semantic):[1]

Search both lexically and semantically

```
lexical_results = db.keyword_search(query)
```

```
semantic_results = db.vector_search(query_embedding)
```

Combine and rerank

```
combined = merge_results(lexical_results, semantic_results)
```

```
reranked = rerank_by_relevance(combined)
```

2. Reranking with cross-encoders:

```
from sentence_transformers import CrossEncoder
```

```
reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-12-v2')
```

Get top 20 from similarity search

```
candidates = vector_db.query(query_embedding, top_k=20)
```

Rerank with cross-encoder

```
scores = reranker.predict([[query, doc['text']] for doc in candidates])
```

```
reranked = [doc for _, doc in sorted(zip(scores, candidates),
```

```
key=lambda x: x[0], reverse=True)][:3]
```

3. Metadata filtering:

Include metadata constraints

```
results = vector_db.query(  
    query_embedding=query_embedding,  
    where_filter={"date": {"$gte": "2024-01-01"}},  
    top_k=5  
)
```

Limits results to recent documents

4. Late chunking strategy:[2]

Traditional:

Document → Chunk → Embed (loses full-document context)

Late chunking:

Document → Full embed → Create chunk embeddings from token embeddings
(preserves full-document semantic context)

5. Query expansion:

Generate multiple query formulations

```
original_query = "renewable energy"  
expanded_queries = [  
    "renewable energy",  
    "solar and wind power",  
    "sustainable electricity",  
    "clean energy sources"  
]
```

Search with all formulations

```
all_results = []  
for q in expanded_queries:  
    q_embedding = embed(q)  
    all_results.extend(vector_db.query(q_embedding, top_k=5))
```

Deduplicate and rerank

Accuracy Assessment

When similarity search works well:

- ✓ Clear semantic match between query and documents
- ✓ Documents in-distribution with training data
- ✓ Sufficient information in single chunks
- ✓ Good embedding model (text-embedding-3-large)

When similarity search struggles:

- ✗ Complex multi-hop reasoning required
- ✗ Out-of-distribution or novel terminology
- ✗ Information split across multiple chunks
- ✗ Typos, abbreviations, domain-specific terms
- ✗ Contradictory or nuanced information

Realistic Accuracy Metrics

Scenario: Retrieve top-3 relevant documents for query

Accuracy metrics:

- MRR@10 (Mean Reciprocal Rank): 75-85%
- Recall@10: 80-90%
- Top-1 accuracy: 60-70%

Improvement with reranking:

- +5-15% accuracy improvement
- Cost: Additional LLM/cross-encoder inference

Best Practices

1. **Benchmark your system:** Don't assume 100% accuracy
2. **Combine approaches:** Hybrid search > single method
3. **Use reranking:** Cross-encoder reranking adds 5-15% accuracy
4. **Monitor failures:** Track queries with poor retrieval
5. **Iterate on chunking:** Adjust chunk size/strategy based on retrieval performance
6. **Use high-quality embeddings:** text-embedding-3-large > text-embedding-3-small for accuracy
7. **Add human feedback:** Collect poor retrieval examples to improve system

References

- [1] OpenAI. (2024). Text Embeddings v3. <https://platform.openai.com/docs/guides/embeddings>
- [2] Firecrawl. (2025). Best chunking strategies for RAG. <https://www.firecrawl.dev/blog/best-chunking-strategies-rag-2025>
- [3] Lakefs. (2025). What is a vector database. <https://lakefs.io/blog/what-is-vector-databases/>
- [4] Instaclustr. (2025). Vector similarity search: Pros, cons, and tips for success. <https://www.instaclustr.com/education/vector-database/>
- [5] Weaviate. (2025). Chunking strategies for RAG. <https://weaviate.io/blog/chunking-strategies-for-rag>
- [6] Pinecone. (2026). OpenAI text embeddings v3. <https://www.pinecone.io/learn/openai-embeddings-v3/>
-

1. a-zA-Z0-9._%+-↩